BCR-6605 – No popup on deviation + deviation on the graph

Goal 1: creating a deviation from the Health settings screen must not show the applicable-flight popup and must not create a new A/C configuration. The popup class itself is not touched.
Goal 2: the deviated threshold must be drawn on the indicator graph.

The six edits at a glance

#	File	Where exactly	Change
1	`WCTreeCellModifier.java`	method `modify(...)`, lines **192**, **217**, **306**	Delete the 3 `markAsDirty(...)` calls
2	`HealthAlarmsPreferenceStore.java`	method `innerSave()`, lines **333–411**	Replace the whole method with an empty one
3	`HealthAlarmsComposite.java`	method `performApply()` – search `wasLastSaveConfirmed`	Remove the `if` around `saveDeviations()`
4	`CreateIndicatorChartModelsJob.java`	import block, after line **300**	Add one import
5	`CreateIndicatorChartModelsJob.java`	after line **334** (the comparator field)	Add 2 fields + `getApplicableDeviation(...)`
6	`CreateIndicatorChartModelsJob.java`	`retrieveThresholdValues(...)`, lines **1079–1120**	Replace the flight-loop body

Line numbers are from the files you sent me. If your copy has drifted, use the FIND block – every FIND below is verbatim from your source, so Eclipse Ctrl+F will land on it.

Why these three edits and not the popup class

The popup and the new configuration are the same code path, in `MgsPreferenceStoreManager.saveAircraftConfigurations()`:

```
        if (!dirtyAircraftsConfigurations.isEmpty()) {
           MgsPreferenceDialogHandler mgsPreferenceDialogHandler = new MgsPreferenceDialogHandler();
           mgsPreferenceDialogHandler.openDialog(dirtyAircraftsConfigurations);      <-- the popup
           ...
                 aircraftBusiness.updateAircraftConfiguration(...);                <-- the configuration
```

`dirtyAircraftsConfigurations` = `AIRCRAFT_CONFIGURATION_HISTORIES` filtered by `DIRTY_AIRCRAFTS`. Make the Health screen stop feeding those two collections and that `if` is false, so **neither the popup nor the configuration happens**. Nothing in `saveAircraftConfigurations()` and nothing in `MgsPreferenceDialogHandler` changes, so the A/C configuration screen keeps working exactly as today. That is the “disconnect” Benoit asked for.

Edit 1 — stop the Health tree marking the aircraft dirty

File: `mgs-client / com.eurocopter.mgs.presentation.configuration.preferences.healthalarms / WCTreeCellModifier.java`
Method: `modify(Object element, String property, Object value)`
Lines: 192, 217 and 306 – three identical occurrences, one at the end of each `if / else if` branch (`WCMechanicalComponentModel`, `WCHealthMechanicalComponentModel`, `WCThresholdModel`)
How to find them: Ctrl+F → `markAsDirty` → 3 hits. Fix all three.

FIND (each of the 3 occurrences):

```
            MgsPreferenceStoreManager.markAsDirty((Aircraft) treeViewer.getData("aircraft"));
            treeViewer.refresh();
```

REPLACE WITH:

```
            // BCR-6605 the settings screen no longer creates an A/C configuration, so it must not
            // mark the aircraft dirty. markAsDirty is what makes
            // MgsPreferenceStoreManager.saveAircraftConfigurations() open the applicable-flight
            // popup and call updateAircraftConfiguration().
            treeViewer.refresh();
```

Keep `treeViewer.refresh()` – that is what redraws the edited cell. After the third one, run Ctrl+Shift+O (Organize Imports) to drop the now-unused `MgsPreferenceStoreManager` import.

Edit 2 — stop the Health store filling the configuration DTO

File: `mgs-client / ...preferences.healthalarms.store / HealthAlarmsPreferenceStore.java`
Method: `innerSave()`, lines **333–411** – the `@Override` immediately after `innerLoad()`
How to find it: Ctrl+O → `innerSave`

Why this is needed even after edit 1: `innerSave()` calls `MgsPreferenceStoreManager.getAircraftConfigurationDto(aircraft)`, and that method *creates and registers* an entry in `AIRCRAFT_CONFIGURATION_HISTORIES`, which `innerSave()` then fills with every threshold on the page before calling `setIndicatorConfigurationUpdated(true)`. With edit 1 alone the popup is gone, but that loaded DTO is still sitting in the map – so the next time the user changes anything on the *A/C configuration* screen for the same aircraft, that screen marks it dirty and every Health edit is written into the new configuration silently. The method must do nothing.

FIND the whole method – it starts here (line 333):

```
   @Override
   public void innerSave() {
      for (Entry<Aircraft, WCMechanicalComponentModel> modelEntry : data.entrySet()) {
         Aircraft aircraft = modelEntry.getKey();
         HealthMonitoringConfiguration configuration;
         AircraftConfigurationHistoryDto aircraftConfigurationHistoryDto;
         try {
            configuration = getConfiguration(aircraft);
            aircraftConfigurationHistoryDto = MgsPreferenceStoreManager.getAircraftConfigurationDto(aircraft);
```

… and ends here (line 411) – everything in between goes too:

```
            aircraftConfigurationHistoryDto.setIndicatorConfigurationUpdated(true);
      }
   }
```

REPLACE the whole method (333–411) WITH:

```
   @Override
   public void innerSave() {
      // BCR-6605 the indicator settings screen no longer creates an A/C configuration.
      // Everything the user changes on this screen is stored as a deviation instead - see
      // saveDeviations() below.
      //
      // The previous body built an AircraftConfigurationHistoryDto. That did two harmful things:
      //   - getAircraftConfigurationDto() REGISTERS the aircraft in
      //     MgsPreferenceStoreManager.AIRCRAFT_CONFIGURATION_HISTORIES, and
      //   - setIndicatorConfigurationUpdated(true) marks it as carrying indicator changes.
      // Either one re-creates a configuration as soon as that aircraft is marked dirty by any
      // other screen. Do not restore it: the A/C configuration screen has its own, untouched path.
   }
```

If you want the old body kept for reference, paste it just above as a block comment – it is the only record of how this screen used to build the DTO. Do not leave it live.

Edit 3 — let the deviation save without a confirmed configuration

> **File:** `mgs-client / ...preferences.healthalarms / HealthAlarmsComposite.java`
> **Method:** `performApply()` (check `performOk()` too)
> **How to find it:** Ctrl+F → `wasLastSaveConfirmed`

This is the edit that breaks silently if you skip it. `MgsPreferenceStoreManager.wasLastSaveConfirmed()` only returns `true` when the user pressed OK on the applicable-flight popup. After edits 1 and 2 that popup never opens from this screen, so `wasLastSaveConfirmed()` is now **permanently false** – and the deviation would never be written, with no error anywhere.

FIND (the guard, shape may differ slightly in your copy):

```
    if (MgsPreferenceStoreManager.wasLastSaveConfirmed()) {
       healthAlarmsPreferenceStore.saveDeviations();
    }
```

REPLACE WITH:

```
    // BCR-6605 the settings screen no longer creates a configuration, so there is no popup
    // confirmation to wait for; wasLastSaveConfirmed() is permanently false here now.
    healthAlarmsPreferenceStore.saveDeviations();
```

Rule: remove every `wasLastSaveConfirmed()` guard that wraps a **deviation** call. Leave any guard that protects something else. If the call is named differently in your copy (`store.saveDeviations()`, `getPreferenceStore().saveDeviations()`...), keep your name – only the `if` and its braces go.

Goal 2 – the deviation on the graph

Every threshold line on the indicator chart is produced by one method, `CreateIndicatorChartModelsJob.retrieveThresholdValues(...)`. It reads the value straight off `FixedThresholdConfiguration` / `LearnedThresholdConfiguration` and knows nothing about deviations – that is why the deviated value never shows.

Precedence used, same as the alarm analyzer in part D: **deviation > relearn request > configuration default**. Date rule, also the same: a deviation applies to a flight when `startDate <= sessionDate` and (`endDate` is null or `endDate >= sessionDate`), where `sessionDate = getHealthStartDate()` falling back to `getUsageStartDate()` – never the download date.

Edit 4 — one import

File: `mgs-client / com.eurocopter.mgs.health.presentation.chart / CreateIndicatorChartModelsJob.java`
Where: the import block, line **300** is the last import

FIND:

```
import com.eurocopter.mgs.presentation.utils.MgsColor;
```

REPLACE WITH:

```
import com.eurocopter.mgs.presentation.utils.MgsColor;
import com.eurocopter.mgs.service.health.common.dto.IndicatorDeviationDto;
```

Everything else this edit needs is already imported in the file: `Date`, `HashMap`, `Map`, `Severity`, `Flight`, `Indicator`, `Aircraft`, `MgsSession`.

Edit 5 — the deviation lookup

File: `CreateIndicatorChartModelsJob.java`
Where: immediately after the `THRESHOLD_VALUES_COMPARATOR` field, line **334** – just before the constructor's javadoc. Nothing is deleted here, this is pure insertion.

FIND (line 334):

```
   private static final Comparator<ThresholdValue> THRESHOLD_VALUES_COMPARATOR = (o1, o2) ->
o1.getBeginningDuration().compareTo(o2.getBeginningDuration());
```

REPLACE WITH:

```
   private static final Comparator<ThresholdValue> THRESHOLD_VALUES_COMPARATOR = (o1, o2) ->
o1.getBeginningDuration().compareTo(o2.getBeginningDuration());

   /** BCR-6605 active deviations of the displayed aircraft, keyed indicator.id + "|" + severity. */
   private final Map<String, IndicatorDeviationDto> deviations = new HashMap<>();

   private boolean deviationsLoaded = false;

   /**
    * BCR-6605 the deviation that governs this threshold for this flight, or null.
    *
    * The key is the INDICATOR id, not the indicator configuration id: the configuration id changes
    * on every Apply, the indicator id does not. Same key as HealthAlarmsPreferenceStore uses.
    *
    * @param indicator
    *           the indicator being charted
    * @param severity
    *           RED or AMBER
    * @param flight
    *           the flight the threshold point belongs to
    * @return the applicable deviation, or null when there is none
    */
   private IndicatorDeviationDto getApplicableDeviation(final Indicator indicator, final Severity severity,
         final Flight flight) {
      if (!deviationsLoaded) {
         // Load once per job, not once per flight
         deviationsLoaded = true;
         Aircraft aircraft = MgsSession.getInstance().getAircraft();
         if (aircraft != null) {
            try {
               for (IndicatorDeviationDto dto : healthBusiness.findActiveDeviations(aircraft.getId())) {
                  deviations.put(dto.getIndicatorId() + "|" + dto.getSeverity(), dto);
               }
            } catch (Exception e) {
               LOGGER.warn("BCR-6605 could not load the deviations for the chart", e);
            }
         }
      }
      IndicatorDeviationDto deviation = deviations.get(indicator.getId() + "|" + severity.name());
      if (deviation == null) {
         return null;
      }
      // The session date of the flight, never the download date
      Date sessionDate = flight.getHealthStartDate() != null ? flight.getHealthStartDate()
            : flight.getUsageStartDate();
      if (sessionDate == null) {
         return null;
      }
      if (deviation.getStartDate() != null && deviation.getStartDate().after(sessionDate)) {
         // Deviation not yet applicable to this flight
         return null;
      }
      if (deviation.getEndDate() != null && deviation.getEndDate().before(sessionDate)) {
         // Deviation already expired for this flight
         return null;
```

```
    }
    return deviation;
}
```

Edit 6 — apply it where the threshold line is built

File: `CreateIndicatorChartModelsJob.java`
Method: `retrieveThresholdValues(Map<Flight, ThresholdConfiguration>, List<FlightModel>, Indicator)`, starts line **1065**
Lines to replace: 1079 to 1120 – the body of the `for (FlightModel flightModel : flightModels)` loop, from `String unitLabel = ...` down to the closing brace of the `else if` block. The lines above it (1066–1078) and below it (1121–1124, the sort and return) are **not** touched.

FIND – this is the complete block, verbatim:

```
        String unitLabel = getUnitLabel(indicator);
        if (thresholdConfiguration instanceof FixedThresholdConfiguration) {
           // Fixed threshold => constant value between beginning and end of configuration
           final Double fixedValue = ((FixedThresholdConfiguration) thresholdConfiguration).getValue();
           double convertedValue = convertThresholdValue(indicator, fixedValue);
           final ThresholdValue thresholdValue = new ThresholdValue(beginningOfFlight, endOfFlight, convertedValue, unitLabel);
           thresholdDataByFlight.get(flight).put(thresholdConfiguration.getThreshold().getSeverity(), thresholdValue);
           values.add(thresholdValue);
        } else if (thresholdConfiguration instanceof LearnedThresholdConfiguration learnedThresholdConfiguration) {

           RelearnThresholdRequest request = getApplicableRelearnRequest(flight, learnedThresholdConfiguration);
           if (request != null) {
              // Request has ended before the flight download
              double value = request.getLearnedValue();
              double convertedValue = convertThresholdValue(indicator, value);
              final ThresholdValue thresholdValue = new ThresholdValue(beginningOfFlight, endOfFlight, convertedValue,
                    learnedThresholdConfiguration.getMinimumThreshold(),
                    learnedThresholdConfiguration.getMaximumThreshold(), unitLabel);
              thresholdDataByFlight.get(flight).put(thresholdConfiguration.getThreshold().getSeverity(), thresholdValue);
              values.add(thresholdValue);
           } else {
              // No started request before the flight
              // Check threshold default value
              if (learnedThresholdConfiguration.getValue() != null) {
                 // Take the default value
                 double value = learnedThresholdConfiguration.getValue();
                 double convertedValue = convertThresholdValue(indicator, value);
                 final ThresholdValue thresholdValue = new ThresholdValue(beginningOfFlight, endOfFlight, convertedValue,
                       learnedThresholdConfiguration.getMinimumThreshold(),
                       learnedThresholdConfiguration.getMaximumThreshold(), unitLabel);
                 thresholdDataByFlight.get(flight).put(thresholdConfiguration.getThreshold().getSeverity(), thresholdValue);
                 values.add(thresholdValue);
              } else {
                 // No default value => do not show the threshold at all
                 final ThresholdValue thresholdValue = new ThresholdValue(beginningOfFlight, endOfFlight,
                       learnedThresholdConfiguration.getMinimumThreshold(),
                       learnedThresholdConfiguration.getMaximumThreshold(), unitLabel);
                 thresholdDataByFlight.get(flight).put(thresholdConfiguration.getThreshold().getSeverity(), thresholdValue);
                 values.add(thresholdValue);
              }
           }
        }
```

REPLACE WITH:

```
        String unitLabel = getUnitLabel(indicator);
        if (thresholdConfiguration == null) {
           // No configuration applicable to this flight - nothing to draw, same as before
           continue;
        }

        // BCR-6605 the deviation applicable to this indicator, severity and flight
        final Severity severity = thresholdConfiguration.getThreshold().getSeverity();
        final IndicatorDeviationDto deviation = getApplicableDeviation(indicator, severity, flight);
        final Double deviatedValue = deviation != null ? deviation.getThresholdValue() : null;
        final Double deviatedMin = deviation != null ? deviation.getMinValue() : null;
        final Double deviatedMax = deviation != null ? deviation.getMaxValue() : null;

        if (thresholdConfiguration instanceof FixedThresholdConfiguration) {
           // Fixed threshold => constant value between beginning and end of configuration
           Double fixedValue = ((FixedThresholdConfiguration) thresholdConfiguration).getValue();
           if (deviatedValue != null) {
              // BCR-6605 the deviation replaces the configured value
              fixedValue = deviatedValue;
           }
           double convertedValue = convertThresholdValue(indicator, fixedValue);
           final ThresholdValue thresholdValue = new ThresholdValue(beginningOfFlight, endOfFlight, convertedValue, unitLabel);
           thresholdDataByFlight.get(flight).put(severity, thresholdValue);
           values.add(thresholdValue);
        } else if (thresholdConfiguration instanceof LearnedThresholdConfiguration learnedThresholdConfiguration) {

           // BCR-6605 the bounds to display: the deviation's when it set them
           final Double min = deviatedMin != null ? deviatedMin : learnedThresholdConfiguration.getMinimumThreshold();
           final double max = deviatedMax != null ? deviatedMax : learnedThresholdConfiguration.getMaximumThreshold();

           RelearnThresholdRequest request = getApplicableRelearnRequest(flight, learnedThresholdConfiguration);
           Double value;
           if (deviatedValue != null) {
              // BCR-6605 a deviated value pins the threshold: it wins over anything learned
              value = deviatedValue;
           } else if (request != null) {
              // Request has ended before the flight download
              value = request.getLearnedValue();
           } else {
              // No started request before the flight - take the threshold default value, if any
              value = learnedThresholdConfiguration.getValue();
           }

           if (value != null) {
              // BCR-6605 clamp only against bounds the DEVIATION supplied, so a chart with no
```

```
                // deviation is drawn exactly as before
                if (deviatedMin != null && value < deviatedMin) {
                   value = deviatedMin;
                }
                if (deviatedMax != null && value > deviatedMax) {
                   value = deviatedMax;
                }
                double convertedValue = convertThresholdValue(indicator, value);
                final ThresholdValue thresholdValue = new ThresholdValue(beginningOfFlight, endOfFlight, convertedValue,
                      min, max, unitLabel);
                thresholdDataByFlight.get(flight).put(severity, thresholdValue);
                values.add(thresholdValue);
             } else {
                // No value at all => do not show the threshold for this flight
                final ThresholdValue thresholdValue = new ThresholdValue(beginningOfFlight, endOfFlight, min, max, unitLabel);
                thresholdDataByFlight.get(flight).put(severity, thresholdValue);
                values.add(thresholdValue);
             }
          }
```

Three behaviour changes, nothing else: the fixed value can come from the deviation, the learned value can be pinned by the deviation, the min/max drawn are the deviation's when it set them. With no deviation every branch produces exactly the same `ThresholdValue` as today. The repeated `thresholdConfiguration.getThreshold().getSeverity()` was hoisted into the local `severity`, which is why the `null` guard at the top is now needed.

Optional — refresh the graph right after Apply

> **File:** `HealthAlarmsComposite.java`, `performApply()`, after the `saveDeviations()` call from edit 3

The job only runs when the chart view is built. If the chart is already open when the deviation is applied, it keeps the old line until the view is refreshed or the flight selection changes. If that is not acceptable, schedule `new UpdateIndicatorChartJob(...)` there. This is a refresh trigger, not part of the overlay – leave it out if you prefer.

Test

#	Action	Expect
1	Rebuild the client, restart	–
2	Health settings: change a threshold value, add a TE reference, Apply	**No applicable-flight popup**. The row turns green
3	`SELECT * FROM public.indicator_deviation ORDER BY start_date DESC LIMIT 5;`	The deviation is there. This is the proof edit 3 worked
4	`SELECT id, creation_date FROM public.aircraft_configuration ORDER BY creation_date DESC LIMIT 5;`	**No new row** for the Apply in step 2
5	A/C configuration screen: change anything, Apply	Popup appears as before, new configuration created. The shared path is untouched
6	Health settings again, then A/C configuration screen, Apply	The new configuration contains **none** of the Health edits. This is the proof edit 2 worked
7	Open the indicator graph for the deviated indicator	The RED / AMBER line sits at the **deviated** value
8	Graph of an indicator with no deviation	Identical to before the change
9	Set the deviation start date in the future, reopen the graph	Line back at the configured value – the date rule holds
10	Acknowledge the deviation, reopen the graph	Line back at the configured value

Decisions taken

Decision	Why	Consequence
Disconnect by removing `markAsDirty` and emptying `innerSave()`, not by touching `saveAircraftConfigurations()` or the popup class	Benoit asked for the settings screen to be disconnected and explicitly warned the popup is still needed when creating a new A/C configuration	Zero risk to the A/C configuration screen
Chart uses `findActiveDeviations` only	Same call the settings screen already makes, so the chart and the settings page can never disagree	An acknowledged (historical) deviation is not redrawn on past flights. If wanted, add `findDeviationHistory` to the same map – the date rule already handles the window
Deviated min/max clamp only when the deviation set them	A configured `maximumThreshold` of 0 means "unset" on some rows; clamping against it would flatten existing charts	None
Deviated value wins over the learned value on the chart	Matches the analyzer, which is what raises the alarm. The line has to sit where the alarm fires	None

Still open after this

Item	Status
Work card and max time without acquisition on the mechanical component	These were saved through the configuration, which this screen no longer creates, so they are now **read-only in practice** from the Health page. `indicator_deviation` has no work card column yet either. Needs Benoit: add a work card column to the deviation, or keep a configuration path for those two fields alone
A deviation turning a FIXED threshold into a LEARNED one, on the chart	Drawn flat at the deviated value. There is no learned configuration row, so there is no relearn request to read a learned value from
Filtering law override during analysis	Stored on the deviation and applied on the settings screen, not yet applied in the analysis chain
Part F (DACOM extraction), part G (purge), Activate & Remove tick box	Untouched